

OpenGL & CUDA-OpenGL Interop Notes for N-Body Simulation

Why OpenGL Interop

Right now every frame you:

1. Run CUDA kernels (positions update on GPU)
2. cudaMemcpy 1M bodies back to CPU (~40MB)
3. Loop through bodies on CPU to build sf::VertexArray
4. SFML uploads that vertex data back to GPU for rendering

With OpenGL interop, positions stay on the GPU the entire time. CUDA writes directly into an OpenGL buffer, and OpenGL renders from that same buffer. Zero CPU involvement in rendering.

OpenGL Basics You Need

The Pipeline (simplified)

Vertex Data → Vertex Shader → Rasterizer → Fragment Shader → Screen

- Vertex Shader: runs per vertex, transforms positions (you'll pass 2D/3D positions)
- Fragment Shader: runs per pixel, decides color
- Rasterizer: converts primitives (points, triangles) to pixels between the two shaders

Key Objects

VAO (Vertex Array Object)

- Stores the configuration of how vertex data is laid out
- Think of it as a "format descriptor" — it remembers which VBO to read from and how to interpret the bytes (stride, offset, type)
- You bind once, configure once, then just bind it again later to draw

VBO (Vertex Buffer Object)

- The actual buffer of vertex data sitting in GPU memory
- This is what CUDA will write into
- For your sim: each entry would be {x, y, z, r, g, b, a} or similar

Shader Program

- Compiled GPU programs written in GLSL
- You need at minimum a vertex shader and fragment shader
- They get compiled at runtime and linked into a "program"

Minimal OpenGL Setup

cpp

```
// Create VAO
GLuint vao;
glGenVertexArrays(1, &vao);
glBindVertexArray(vao);

// Create VBO
GLuint vbo;
glGenBuffers(1, &vbo);
glBindBuffer(GL_ARRAY_BUFFER, vbo);
glBufferData(GL_ARRAY_BUFFER, N * sizeof(float) * 6, nullptr, GL_DYNAMIC_DRAW);
// GL_DYNAMIC_DRAW tells OpenGL this buffer will be updated frequently

// Define vertex layout
// Position: 3 floats starting at offset 0
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);

// Color: 3 floats starting at offset 12 bytes
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)(3 * sizeof(float)));
glEnableVertexAttribArray(1);
```

Minimal Shaders

Vertex shader:

glsl

```
#version 330 core
layout(location = 0) in vec3 aPos;
layout(location = 1) in vec3 aColor;

out vec3 vertColor;
uniform mat4 projection;

void main() {
    gl_Position = projection * vec4(aPos, 1.0);
    gl_PointSize = 1.0;
    vertColor = aColor;
}
```

Fragment shader:

```
glsl

#version 330 core
in vec3 vertColor;
out vec4 FragColor;

void main() {
    FragColor = vec4(vertColor, 1.0);
}
```

Drawing

```
cpp

glClear(GL_COLOR_BUFFER_BIT);
glUseProgram(shaderProgram);
glBindVertexArray(vao);
glDrawArrays(GL_POINTS, 0, N);
// swap buffers (depends on windowing library)
```

Windowing Library

SFML uses its own OpenGL context internally but doesn't expose the raw VBO/VAO workflow you need for interop. You have three options:

Option 1: GLFW + GLAD/GLEW (recommended)

- GLFW handles window creation, input, and OpenGL context
- GLAD or GLEW loads OpenGL function pointers (OpenGL functions aren't linked statically — you need a loader)
- This is the standard combo used in every OpenGL tutorial
- You'll lose SFML but gain full control

Option 2: SFML with raw OpenGL calls

- SFML creates an OpenGL context for you, so you CAN make raw GL calls
- But you have to be careful not to conflict with SFML's internal GL state
- Awkward and fragile, not recommended for interop

Option 3: SDL2 + GLEW

- Similar to GLFW, just a different windowing library
- Either works fine

Go with GLFW + GLAD. Here's what the window setup looks like:

cpp

```
#include <glad/glad.h>
#include <GLFW/glfw3.h>

glfwInit();
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

GLFWwindow* window = glfwCreateWindow(2560, 1440, "nbody sim", nullptr, nullptr);
glfwMakeContextCurrent(window);
gladLoadGLLoader((GLADloadproc)glfwGetProcAddress);

glEnable(GL_PROGRAM_POINT_SIZE);
```

CUDA-OpenGL Interop

How It Works

1. Create an OpenGL VBO normally

2. Register that VBO with CUDA (tells CUDA "I want to write to this buffer")
3. Each frame: a. Map the buffer (CUDA gets a device pointer to the VBO's memory) b. Run your CUDA kernel writing positions/colors into that pointer c. Unmap the buffer (hands it back to OpenGL) d. OpenGL draws from the same buffer

The data never leaves the GPU.

Setup

cpp

```
#include <cuda_gl_interop.h>

// IMPORTANT: call this BEFORE any CUDA operations
// It tells CUDA which GPU to use and that it will share with OpenGL
cudaGLSetGLDevice(0);

// Create OpenGL VBO
GLuint vbo;
glGenBuffers(1, &vbo);
glBindBuffer(GL_ARRAY_BUFFER, vbo);
glBufferData(GL_ARRAY_BUFFER, N * sizeof(float) * 6, nullptr, GL_DYNAMIC_DRAW);

// Register with CUDA
cudaGraphicsResource* cudaVBO;
cudaGraphicsGLRegisterBuffer(&cudaVBO, vbo, cudaGraphicsMapFlagsWriteDiscard);
// WriteDiscard means CUDA will overwrite the entire buffer each frame
// (no need to preserve old data)
```

Per-Frame Usage

cpp

```

// 1. Map the buffer – CUDA gets access
float* d_vboPtr;
size_t bufferSize;
cudaGraphicsMapResources(1, &cudaVBO, 0);
cudaGraphicsResourceGetMappedPointer((void**)&d_vboPtr, &bufferSize, cudaVBO);

// 2. Run kernel that writes positions and colors into d_vboPtr
writeToVBO<<<numBlocks, threadsPerBlock>>>(d_bodies, d_vboPtr, N);

// 3. Unmap – hand back to OpenGL
cudaGraphicsUnmapResources(1, &cudaVBO, 0);

// 4. OpenGL draws
glClear(GL_COLOR_BUFFER_BIT);
glBindVertexArray(vao);
glDrawArrays(GL_POINTS, 0, N);
glfwSwapBuffers(window);

```

The Kernel

You need a small kernel that copies position and color data from your body array into the VBO's interleaved format:

```

cuda

__global__ void writeToVBO(body* bodies, float* vbo, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= N) return;

    int offset = idx * 6; // 6 floats per vertex: x, y, z, r, g, b

    // Position
    vbo[offset + 0] = bodies[idx].pos.x;
    vbo[offset + 1] = bodies[idx].pos.y;
    vbo[offset + 2] = bodies[idx].pos.z;

    // Color – depth-based like your current SFML code
    float depth = fminf(fmaxf(0.2f + bodies[idx].pos.z * 0.4f, 0.2f), 1.0f);
    vbo[offset + 3] = depth; // R
    vbo[offset + 4] = depth; // G
    vbo[offset + 5] = 1.0f; // B
}

```

VBO Data Layout

You have two options for how to organize data in the VBO:

Interleaved (recommended for your case):

```
[x, y, z, r, g, b, x, y, z, r, g, b, ...]
```

One vertex's data is contiguous. Better cache behavior when rendering.

Stride = 6 * sizeof(float) = 24 bytes.

Separate buffers:

```
Position VBO: [x, y, z, x, y, z, ...]  
Color VBO:    [r, g, b, r, g, b, ...]
```

Two VBOs, two CUDA registrations. More flexible but more overhead.

Go with interleaved.

Projection Matrix

Your current code maps positions to screen pixels manually:

```
cpp  
  
float px = centerPx + bodies[i].pos.x * Scale;
```

In OpenGL, the vertex shader handles this via a projection matrix. OpenGL's coordinate system goes from -1 to 1 (NDC — Normalized Device Coordinates), so you need a matrix that maps your AU range to that.

For 2D (orthographic):

```
cpp
```

```
// Maps x: [-maxRadius, maxRadius] and y: [-maxRadius, maxRadius] to screen
// You can build this matrix manually:
float left   = -maxRadius;
float right  =  maxRadius;
float bottom = -maxRadius;
float top    =  maxRadius;
float near   = -100.0f;
float far    =  100.0f;

float ortho[16] = {
    2.0f/(right-left), 0, 0, 0,
    0, 2.0f/(top-bottom), 0, 0,
    0, 0, -2.0f/(far-near), 0,
    -(right+left)/(right-left), -(top+bottom)/(top-bottom), -(far+near)/(far-near),
};

// Upload to shader
GLint projLoc = glGetUniformLocation(shaderProgram, "projection");
glUniformMatrix4fv(projLoc, 1, GL_FALSE, ortho);
```

For 3D with perspective (for later):

```
cpp

// perspective projection + a view/camera matrix
// Look into GLM library for mat4, perspective(), lookAt()
```

Camera Controls (for 3D)

Once you're in 3D, you'll want to rotate/zoom the view. This requires a view matrix.

GLM library makes this easy:

```
cpp
```



```

#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>

glm::mat4 view = glm::lookAt(
    glm::vec3(0, 0, 20),    // camera position
    glm::vec3(0, 0, 0),    // look at center
    glm::vec3(0, 1, 0)     // up direction
);

glm::mat4 projection = glm::perspective(
    glm::radians(45.0f),    // FOV
    2560.0f / 1440.0f,     // aspect ratio
    0.1f, 1000.0f          // near/far clip planes
);

glm::mat4 mvp = projection * view;
glUniformMatrix4fv(mvpLoc, 1, GL_FALSE, &mvp[0][0]);

```

For interactive rotation, track mouse input via GLFW callbacks and update the camera position each frame.

What Changes in Your Code

Remove

- SFML entirely (window, VertexArray, draw calls, event loop)
- The CPU-side memcpy for rendering (the one after cudaDeviceSynchronize)
- The CPU loop that builds the vertex array

Keep

- All CUDA kernels (calcAccel, leapfrog, etc.)
- The CPU tree build and COM pass (until you move those to GPU)
- The body array and device body array
- The memcpy for tree building (CPU still needs positions for tree build)

Add

- GLFW + GLAD for windowing
- OpenGL VBO, VAO, shaders

- CUDA-OpenGL interop registration
- writeToVBO kernel
- Shader compilation code
- Projection matrix setup

The Main Loop Becomes

cpp

```
while (!glfwWindowShouldClose(window)) {
    glfwPollEvents();

    // Physics (unchanged)
    leapfrog(d_bodies, bodies, d_nodes, nodes, nodeCount, numBlocks, threadsPerBlock);

    // Rendering (new)
    cudaGraphicsMapResources(1, &cudaVBO, 0);
    cudaGraphicsResourceGetMappedPointer((void**)&d_vboPtr, &bufferSize, cudaVBO);
    writeToVBO<<<numBlocks, threadsPerBlock>>>(d_bodies, d_vboPtr, N);
    cudaGraphicsUnmapResources(1, &cudaVBO, 0);

    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(shaderProgram);
    glBindVertexArray(vao);
    glDrawArrays(GL_POINTS, 0, N);
    glfwSwapBuffers(window);
}
```

Note: you still need the CPU memcpy inside leapfrog() for tree building.

The interop only eliminates the RENDERING memcpy. The tree-build memcpy stays until you move tree construction to the GPU.

Build Command

sh

```
nvcc nbody.cu -o nbody.exe -std=c++17 -diag-suppress 1394 -diag-suppress 1388 \
-I "path/to/glad/include" \
-I "path/to/glfw/include" \
-I "path/to/glm" \
-L "path/to/glfw/lib" \
-lglfw3 -lopengl32 -lgdi32
```

You'll also need to compile glad.c (it's a single C file) and link it in.

Libraries to Download

1. **GLFW**: <https://www.glfw.org/download> — pre-compiled Windows binaries
 2. **GLAD**: <https://glad.dav1d.de/> — web generator
 - Language: C/C++
 - GL version: 3.3
 - Profile: Core
 - Generate and download, gives you glad.c + glad.h
 3. **GLM** (for 3D math, optional for now): <https://github.com/g-truc/glm>
 - Header-only, just add the include path
-

Common Gotchas

- Call `cudaGLSetGLDevice(0)` BEFORE any other CUDA calls. If you've already called `cudaMalloc`, you need to restructure initialization order.
- The OpenGL context must exist before you register buffers with CUDA. So: create window → create VBO → register with CUDA → then `cudaMalloc` bodies.
- Map/unmap must happen every frame. Don't leave a buffer mapped while OpenGL tries to draw from it — undefined behavior.
- `glBufferData` with `nullptr` allocates the buffer without filling it. CUDA will fill it each frame.
- If you see a black screen, check: is the shader compiled? Is the projection matrix correct? Are positions in the right range? Is `GL_PROGRAM_POINT_SIZE` enabled?
- CUDA errors from interop functions are often silent. Wrap every CUDA call in error checking during development:

cpp

```
#define CUDA_CHECK(x) { cudaError_t err = x; if (err != cudaSuccess) \
    printf("CUDA error %s at %s:%d\n", cudaGetErrorString(err), __FILE__, __LINE__
```

Initialization Order

1. glfwInit() + create window + gladLoadGL
2. Create VBO and VAO (OpenGL)
3. Compile shaders (OpenGL)
4. cudaGLSetGLDevice(0)
5. Register VBO with CUDA
6. cudaMalloc for bodies, nodes
7. Initialize bodies, build tree, compute COM
8. cudaMemcpy bodies and nodes to device
9. Enter main loop